

Course : Individual Software Development Process' 2026

Iteration Report : Iteration 3

***By* Ultrasmooth**

Chosen Irl Challenge : Project A - Lab Data Management



Jirat Kiattrairong 6810545506

Napakhet Namsrioun 6810545727

Panyasiri Aimngern 6810545743

Panisara Niyathirakul 6810545751

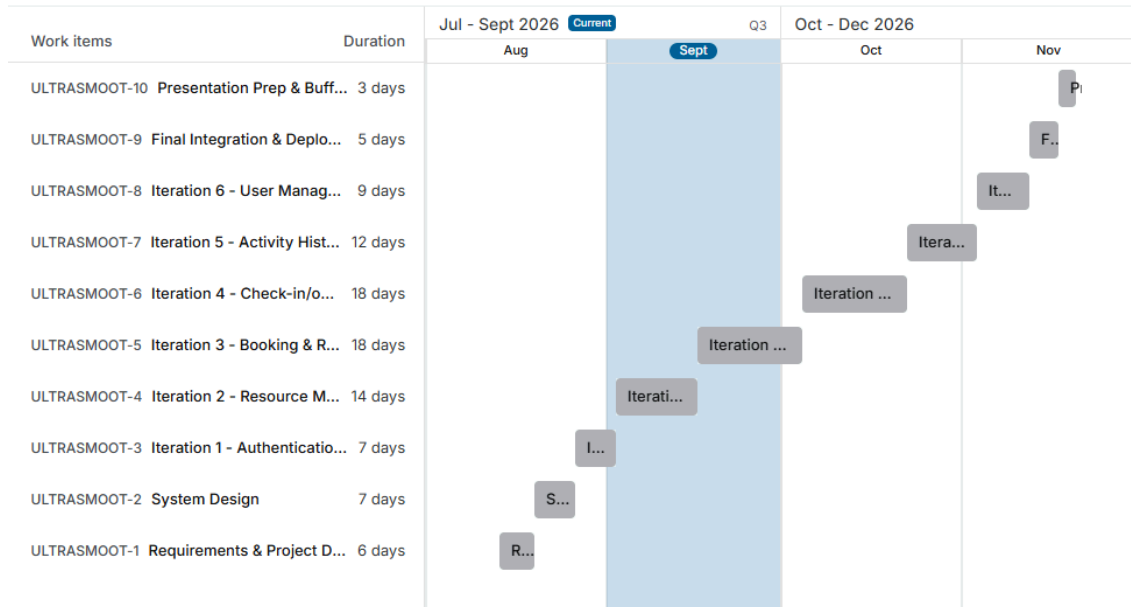
Table of Contents

1. Project Snapshot.....	1
1.1 Project-Wide Gantt Chart.....	1
1.2 Current Iteration Gantt Chart.....	2
2. Sprint/Board Status Snapshot.....	3
3. Scope Addition/Change Log.....	5
4. Retrospective.....	5
5. Testing & Quality Notes.....	6
6. Project Risk Register.....	7
7. Individual Contribution Log.....	8

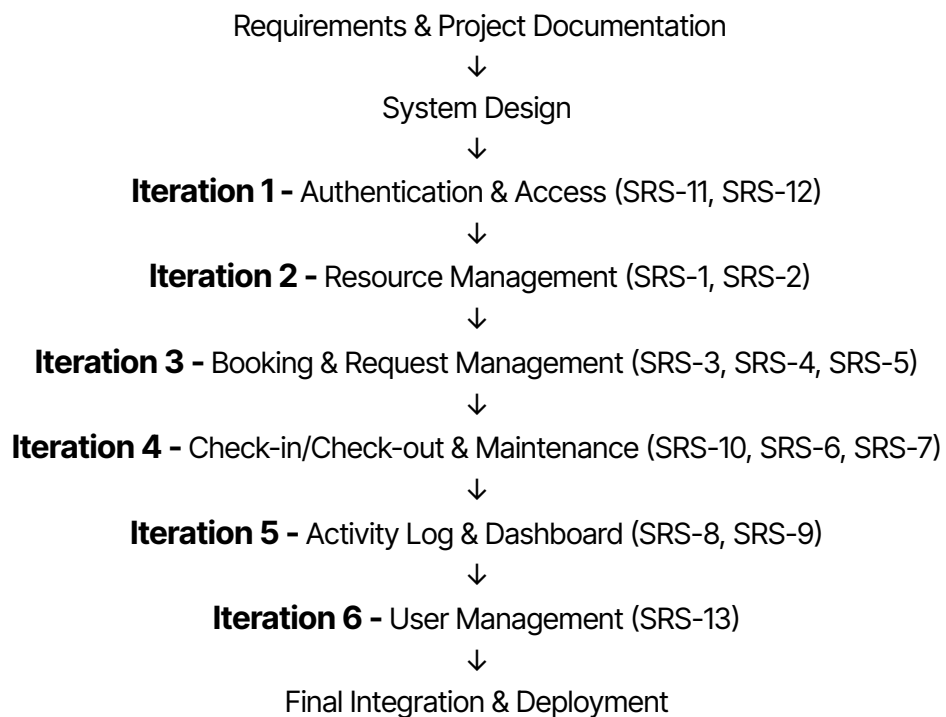
1. Project Snapshot

Project Repository : <https://github.com/Ultrasmoot/Ultrasmooth>

1.1 Project-Wide Gantt Chart

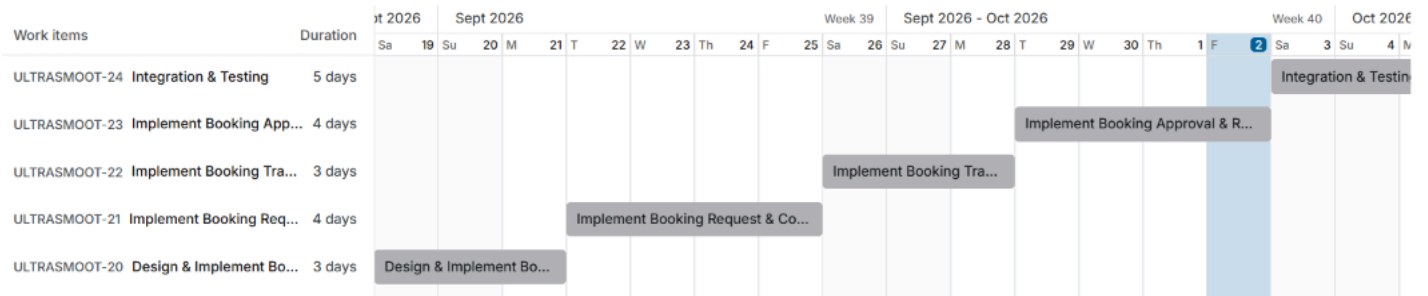


Critical Path :

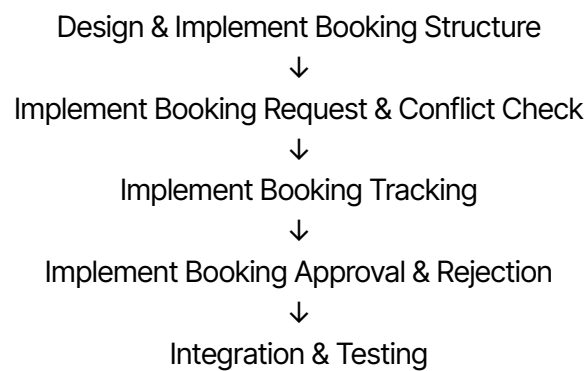


Critical Path Duration : 99 calendar days

1.2 Current Iteration Gantt Chart



Critical Path :



Critical Path Duration : 19 calendar days

2. Sprint/Board Status Snapshot

Iteration Goal : The main goal of Iteration 3 was to deliver a working booking and request flow for the VASE Lab Data Management System. Students can submit a booking request for a lab resource and follow its status. Administrators can review pending requests and approve or reject each one. The system blocks any request that overlaps with another booking for the same resource. This supports Sub-Goals SG-2 (Booking & Request Management) and SG-5 (avoiding conflicting bookings) from our SRS, through SRS-3, SRS-4 and SRS-5.

During this iteration, the team focused on three user stories :

- US-3 : Submit a booking request
- US-4 : Track my bookings/requests
- US-5 : Review and approve/reject requests

The implemented features include booking validation (required fields, valid dates, end time after start time, no start time in the past), a resource availability check, an overlap check for the same resource, a "my requests" list with status badges, a Pending list for administrators, and approve/reject with a check for requests that have already been decided.

Capacity Declared : 3 user stories (US-3, US-4 and US-5). In Iterations 1 and 2 we declared 2 stories each, so this is one more than before. We chose these three together because they share the same bookings table and Booking Model, and the work could be split among the four members. US-4 and US-5 also cannot be used or tested without US-3.

User Story	Description	Owner	Status	Within Capacity (Y/N)	Notes
US-3	As a Ph.D. Student or Undergraduate Student, I want to submit a booking/request so I can reserve a resource.	Napakhet Namsrioun (backend) Panisara Niyathirakul (booking form) Jirat Kiattrairong (booking table)	Done	Y	The student chooses a resource, start time, end time and purpose. The system rejects empty fields, an end time that is not after the start time, a start time in the past, and a purpose longer than 255 characters. It also rejects a resource that does not exist, is archived, or is not available. A request is blocked if a Pending or Approved booking overlaps on the same resource (SRS-3). A Rejected booking does not block the slot, and back-to-back bookings are allowed. The form shows the error message

					returned by the server. Only Ph.D. and Undergraduate students can submit; other roles are refused.
US-4	As a Ph.D. Student or Undergraduate Student, I want to track my bookings/requests so I know their status.	Panisara Niyathirakul	Done	Y	Students can view their own booking requests with Pending, Approved or Rejected status badges. The list is sorted by start time, latest first. Booking data, including resource names and current statuses, is read from the database. Students can refresh the list manually, and the page also reloads it automatically every 30 seconds while the Requests page is open and visible.
US-5	As a Lab Administrator, I want to review and approve/reject requests so I can control resource usage.	Panyasiri Aimngern	Done	Y	Administrators can view Pending booking requests, including the resource name, requester name and requester email, sorted by start time with the earliest first. Administrators can approve or reject a Pending request, and the system records the decision, the administrator ID and the decision time. A request that has already been decided cannot be decided again. The update only applies while the status is still pending, so two administrators acting at the same moment cannot overwrite each other.

3. Scope Addition/Change Log

Item added/changed	Reason	What as dropped to compensate (one-in-one-out)
Announcements : Professors and Administrators can post announcements. All logged-in users can read them on the home page, with pinned announcements shown first. Professors and Administrators can pin or unpin any announcement, and only the author can delete one.	Added as a small extension to support communication among lab users, so that important information can be published and pinned.	None. This was a small extension, and no existing user story was removed or reduced.

4. Retrospective

What went well	What didn't go well	Action items for next sprint
The Booking Model holds all booking rules in one place: input validation, resource availability, and the overlap check. Booking creation runs in a database transaction and locks the resource row, which reduces the risk of conflicting bookings being saved at the same time. Approval and rejection only update a booking that is still Pending. Booking lists now read resource and requester details directly from the database.	The booking workflow still has no dedicated automated tests. The existing tests in test_security.py cover only authentication, rate limiting, signup and reset-password validation, and configuration. We are still testing for overlapping and back-to-back bookings, rejected bookings, resource status checks, invalid and timezone-offset dates, purposes longer than 255 characters, concurrent requests and deciding a request that has already been decided.	Before Iteration 4 starts, write automated tests for booking validation, the overlap check (partial overlap, back-to-back, a rejected booking freeing the slot), resource availability (In Use, Maintenance, archived), concurrent requests, and approve/reject including an already-decided request.
The booking and announcement endpoints use the shared guards from Iteration 1, so roles are checked on the server and not only by hiding buttons. The user stories were shared among members, and we agreed on the order in which shared files were pushed, so the work could run in parallel.	Professors do not have a separate read-only view of all booking requests, and the access level they need has not been confirmed by the stakeholder.	Professors need a read-only view of all booking requests and whether administrators should see requests in all statuses. Also review the announcement permissions with them.

The database accepts only the three booking status values (Pending, Approved, Rejected). The application and database timezone is now set to Asia/Bangkok, so time checks and decision timestamps are consistent.	The rate limiter keeps its counters in memory, so they are per process and reset when the server restarts. This is not enough for a deployment with several workers.	Before final deployment, replace the in-memory rate limiter with a shared store such as Redis.
---	--	--

5. Testing & Quality Notes

Note : The team reviewed the implementation of US-3, US-4, US-5 and the announcement feature. The Booking Model was reviewed for input validation, resource availability, conflict detection and transaction handling.

Automated security regression tests are in backend/tests/test_security.py. They cover Google sign-in validation, rate limiting, signup input validation, password-reset token handling and production secret-key configuration. These tests do not cover the booking workflow. Dedicated tests are still needed for overlapping bookings, back-to-back bookings, rejected bookings, resource status validation, concurrent booking requests, and approval or rejection of requests that have already been decided.

For US-3, Validation rejects missing or blank required fields, invalid dates, end times that are not after start times, start times in the past, and purposes longer than 255 characters. The backend also checks that the selected resource exists, is not archived, and has an Available status. Date/time values with a timezone offset are converted to server time, and values without an offset are treated as server-local time. The app and the database are set to Asia/Bangkok, so the "start time in the past" check and the decision time use the same timezone. Only Pending and Approved bookings count as conflicts, so back-to-back bookings are allowed and Rejected bookings do not block a time slot.

For US-4, The system returns only the bookings that belong to the logged-in user. Statuses are loaded when the Requests page opens, and the page reloads them every 30 seconds while it is open and visible.

For US-5, Only administrators can open the Pending list and the decided endpoint. The system accepts only "approve" or "reject" as a decision. A request that has already been decided cannot be decided again, even if two administrators act at the same moment, because the status is updated only while it is still pending.

For announcements, The system rejects announcements with an empty title or body. Only the author can delete an announcement.

The seed file includes three sample bookings (Pending, Approved and Rejected) and a demo student account, so the booking workflow can be demonstrated.

6. Project Risk Register

Risk ID	Description	Likelihood (L/M/H)	Impact (L/M/H)	Mitigation/Contingency	Status
R-01	Double booking. If the overlap check and the insert were separate steps, two simultaneous requests could both be saved for one slot.	L	H	Booking creation now locks the selected resource row and runs the conflict check and the insert in one database transaction. Add automated concurrency tests to confirm that simultaneous requests for overlapping slots cannot create conflicting bookings.	Open
R-02	The Requests page shows resource names and requester details from the database. Administrators can see only pending requests, and professors have no separate read-only view of all booking requests. The access level each role needs is not yet confirmed.	M	M	Confirm with the stakeholder whether professors need a read-only view of all booking requests. Update permissions and views based on the agreed requirements.	Open
R-03	The booking system checks resource availability before accepting a request. Only Available resources can be booked, and archived resources are rejected. These checks are not yet tested against resources that are In Use or under Maintenance.	M	M	Add automated tests that confirm requests are rejected when the resource is In Use, under Maintenance, or archived. Consider showing resource availability in the booking form.	Open
R-04	The rate limiter keeps counters in memory, so they are per process and reset on restart. It gives weaker protection when several workers run.	M	M	Use a shared store such as Redis or Flask-Limiter before final deployment.	Open

R-05	Date/time values without a timezone offset are treated as server-local time. If the server timezone is changed or misconfigured, the "start time in the past" check and decision timestamps would be wrong.	L	M	Keep TZ=Asia/Bangkok in the Dockerfile and docker-compose settings, and add a test for date/time handling.	Monitoring
------	---	---	---	--	------------

7. Individual Contribution Log

Student Name	Tasks owned this sprint	Course concept(s) applied	Evidence	Self-rated effort (1-5)
Napakhet Namsrioun	US-3 : wrote the booking validation (required fields, valid dates, end time after start time, no start time in the past), the overlap check against Pending and Approved bookings, the create endpoint and the response format. Announcements : wrote the validation, the post endpoint and the response format. Security : wrote the Google token verification and GoogleAuthTests.	Business rules in the Model layer (MVC) : All validation, the resource availability check and the conflict check are in the Booking Model, and the controller only passes data and returns the response. Interval overlap : two bookings overlap when each one starts before the other ends, so back-to-back bookings are allowed. Concurrency control : a booking is created inside a transaction that locks the resource row, so two simultaneous requests cannot take the same slot. Safe database access : queries use parameters instead of putting user input into the SQL text. Server-side verification : the server verifies the Google sign-in token and does not trust the data sent by the browser.	6a6144e Create booking controller and add API 02e8f2f Update booking_model.py d6ab534 Add announcement validation and creation functions d12caa0 Add Google credential verification e9505e1 Add Google authentication tests	5

Panisara Niyathirakul	US-4 : wrote the "my bookings" query and endpoint. Requests page: wrote the markup, table, status badges, and the loading and display of the list. US-3 form : wrote the booking request form and the error message shown when a request is blocked. Announcements : wrote the list, get and delete functions (author only), the announcements panel on the Home page, and the new-announcement form. .	Data access control : the "my bookings" query filters by the logged-in user's ID, so a student cannot see other people's bookings. Ownership check : only the author can delete an announcement, and the server enforces this. Separation of concerns (MVC) : the backend returns data and the frontend only displays it. User feedback : validation messages from the server are shown in the booking form, so the student knows what to fix. Output escaping : text written by users, such as the purpose and the announcement text, is escaped before it is displayed, which prevents script injection (XSS).	ff69553 Add frontend booking request b76d087 Add new announcement box 47e8910 Replace sign-in with announcement box 2b1a3ea Get and pin announcement functions in announcement_model.py a80f361 Add delete function for announcement's author	5
Jirat Kiattrairong	Database : designed the bookings and announcements tables and added three sample bookings and a demo student to the seed file. Backend : connected the booking and announcement routes in app.py. Frontend : wrote the role-based visibility rules and the opening tag of the Requests section. Security and configuration wrote the rate limiter and applied it to the auth endpoints, the SECRET_KEY and APP_ENV check in config.py, and the	Relational database design : foreign keys link each booking to a real resource and user, and decided_by links to the administrator who made the decision. The status is an ENUM with the default value Pending, so only valid values can be saved. Role-based access control : the interface changes by role, and the server checks the role again, so hiding a button is never the only protection. Rate limiting : a sliding-window limiter on the login, signup and password-reset endpoints reduces brute-force	6976414 Add example env for forget email and secure secret-key a346826 Add env and use for Bangkok timezone 205c480 Update auth_middleware.py	5

	timezone settings in the Dockerfile and docker-compose. Tests : created test_security.py, RateLimitTests and ConfigTests.	attempts and abuse. Secure configuration : settings come from environment variables, and in production the app refuses to start if SECRET_KEY is missing or still a placeholder. Time consistency : the app and the database use the same timezone (Asia/Bangkok), so the "start time in the past" check and the decision time agree.		
Panyasiri Aimngern	US-5 : wrote the Pending list query (earliest start time first), the approve/reject logic with the "already decided" check, the decide endpoint, and the click handler for the Approve and Reject buttons. Shared code : wrote the common query that returns resource and requester names. Announcements : wrote the pin/unpin function, the pin endpoint, and the click handler for the Pin and Delete buttons. Tests : wrote ValidationTests.	State transition rules : a request can change from Pending to Approved or Rejected only once, and the system checks this before saving. Compare-and-set update : the status is updated only while it is still Pending, so two administrators who decide at the same moment cannot overwrite each other. Audit data : each decision saves who decided and when. Role-based access control on the server : The Pending list and the decide endpoint are for administrators only. Code reuse (DRY) : one shared query returns resource and requester names for every booking list, so all views show the same data.	b47979c Implement US-5 (pending queue, approve/reject requests) 4528aed Implement booking model (pending list, admin history, approval, rejection) f9e9408 Add request decision, pin announcements 8f0b99f Add pin function bc8dd9f User model add text validation c94c27c Create announcement model	5